Golang_Ransomware | Write-Up by Wh0isGOTH4M

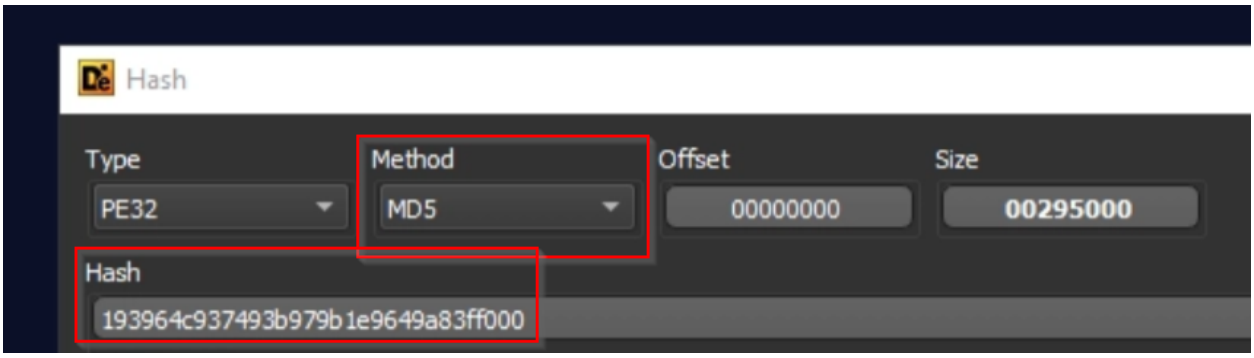
Sherlock Scenario

> Over the past few years, malware researchers have noticed a worrying trend: malicious groups are increasingly using Go, a powerful, easy-to-use, and platform-independent programming language, to create stealthy and resilient malware.
>
> Unfortunately, a new strain of Golang malware has been detected on your organization's network. Initial reports indicate that traditional antivirus solutions are struggling to detect this malware, which uses sophisticated evasion techniques. Upon further investigation, it becomes clear that you need to understand Golang's subtleties.
>
> You need to dissect the inner workings of this Golang malware, identify its capabilities, uncover its propagation methods, and analyze its functions.

Difficulty: Hard

Task 0 : What is the MD5 hash value of the malware detected?

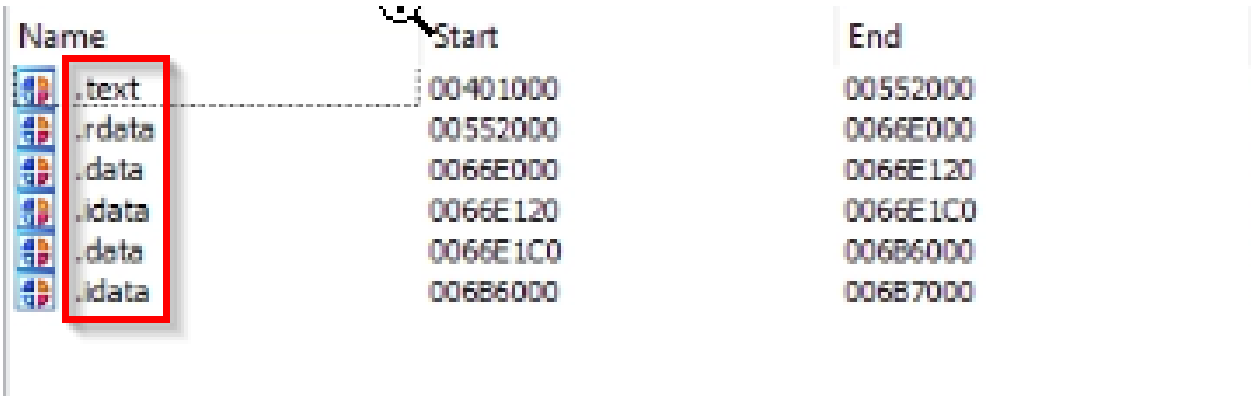
In my analysis of the malware binary using Detect It Easy, I examined the file's structural characteristics and cryptographic checksums. As highlighted in the hash properties window, I selected the **MD5** method to uniquely identify the payload file.



Answer: 193964c937493b979b1e9649a83ff000

Task 1 : How many sections are present in this sample?

While reviewing the PE section analysis of the malware binary in IDA Pro, I inspected the structural makeup of the executable file. As highlighted in the section table, I found that the binary contains a total of **6** distinct sections

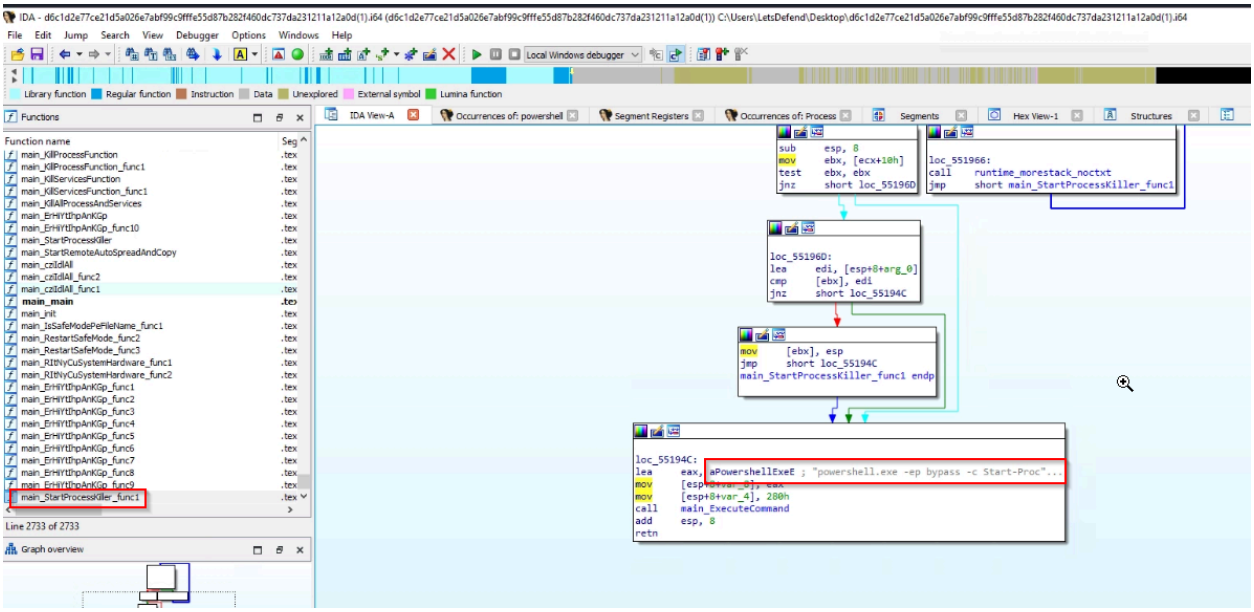


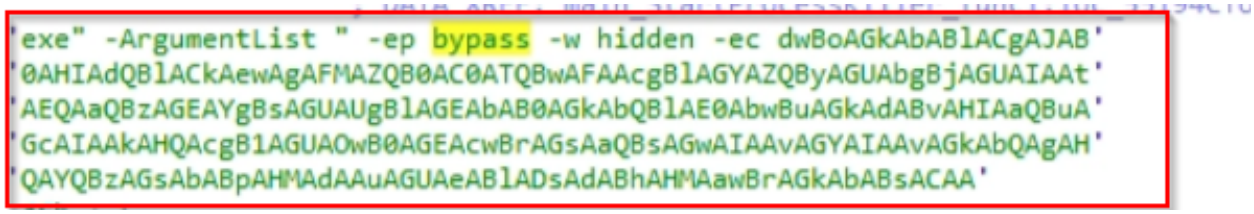
Answer: 6

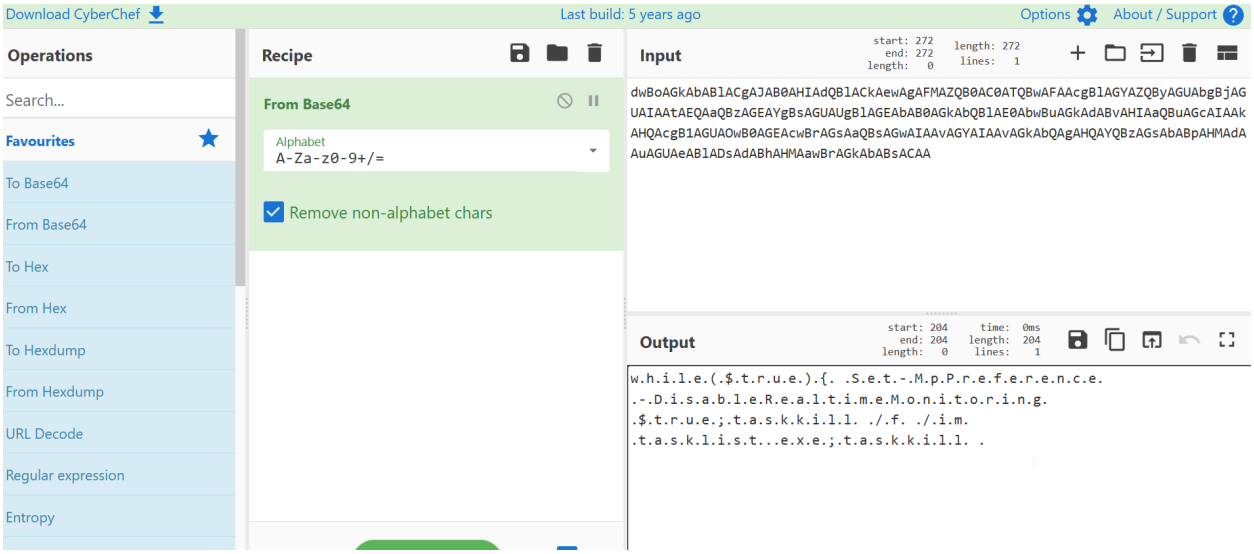
Task 2 : What code does the malware execute to kill the processes?

During my analysis of the embedded execution strings, I decoded a Base64 payload using CyberChef to uncover its persistence and defense-evasion

commands. As highlighted in the script output, I observed that it executes a continuous loop to disable real-time defense mechanisms and terminate monitoring utilities.



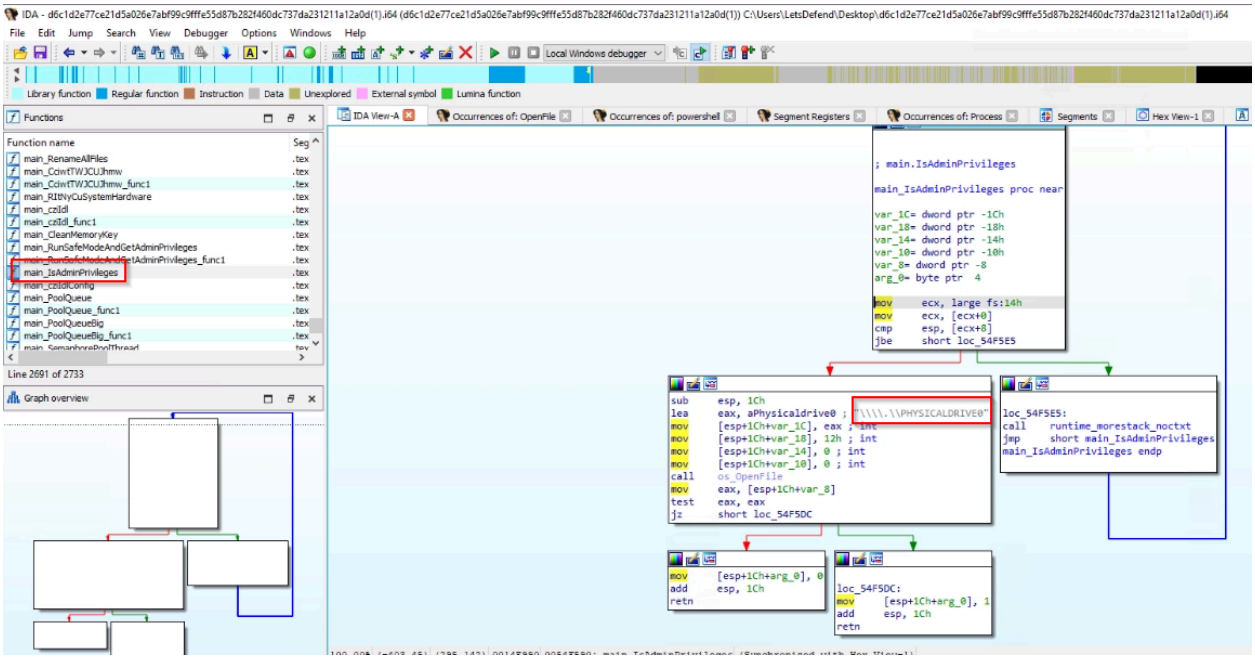




Answer: **while($true){ Set-MpPreference -DisableRealtimeMonitoring $true;taskkill /f /im tasklist.exe;taskkill**

Task 3 : What is the first parameter of the OpenFile function in the function that grants the malware high privileges?

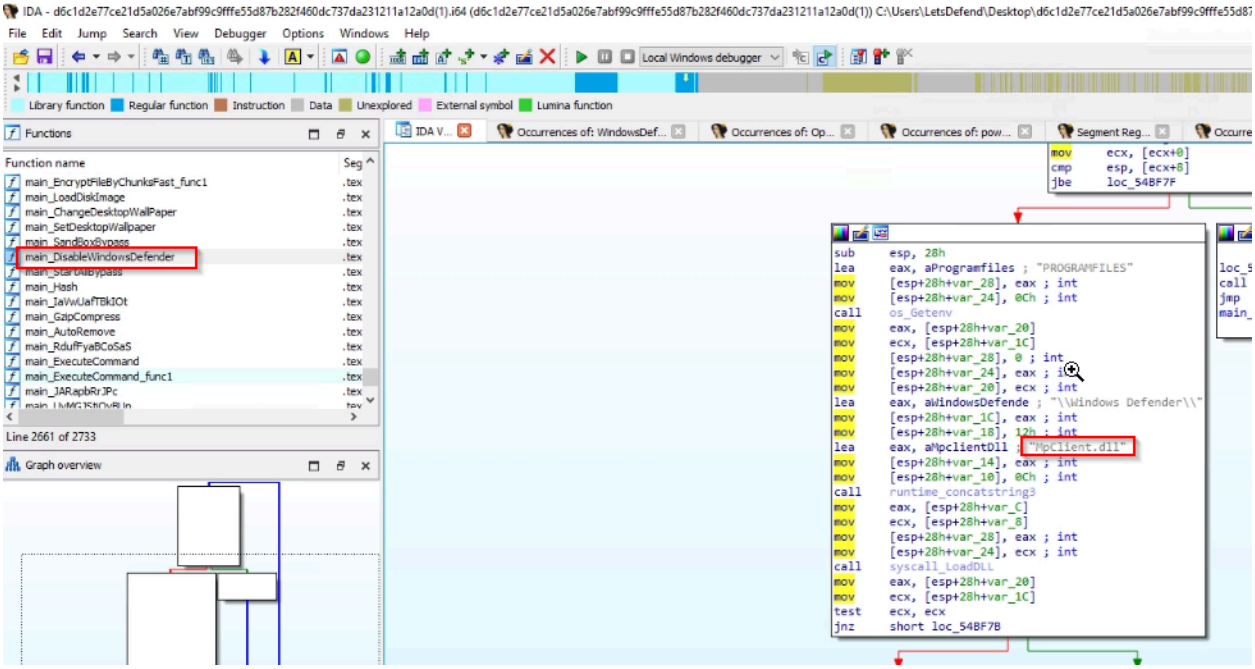
In my review of the privilege-escalation function (`main_IsAdminPrivileges`), I looked at how the malware prepares low-level system operations. As highlighted in the argument setup block within IDA Pro, I saw that the `OpenFile` function is passed `\\.\PHYSICALDRIVE0` as its first parameter to obtain direct physical drive access.



Answer: \\\\.\\PHYSICALDRIVE0

Task 4 : What is the name of the DLL used by the malware to disable Windows Defender?

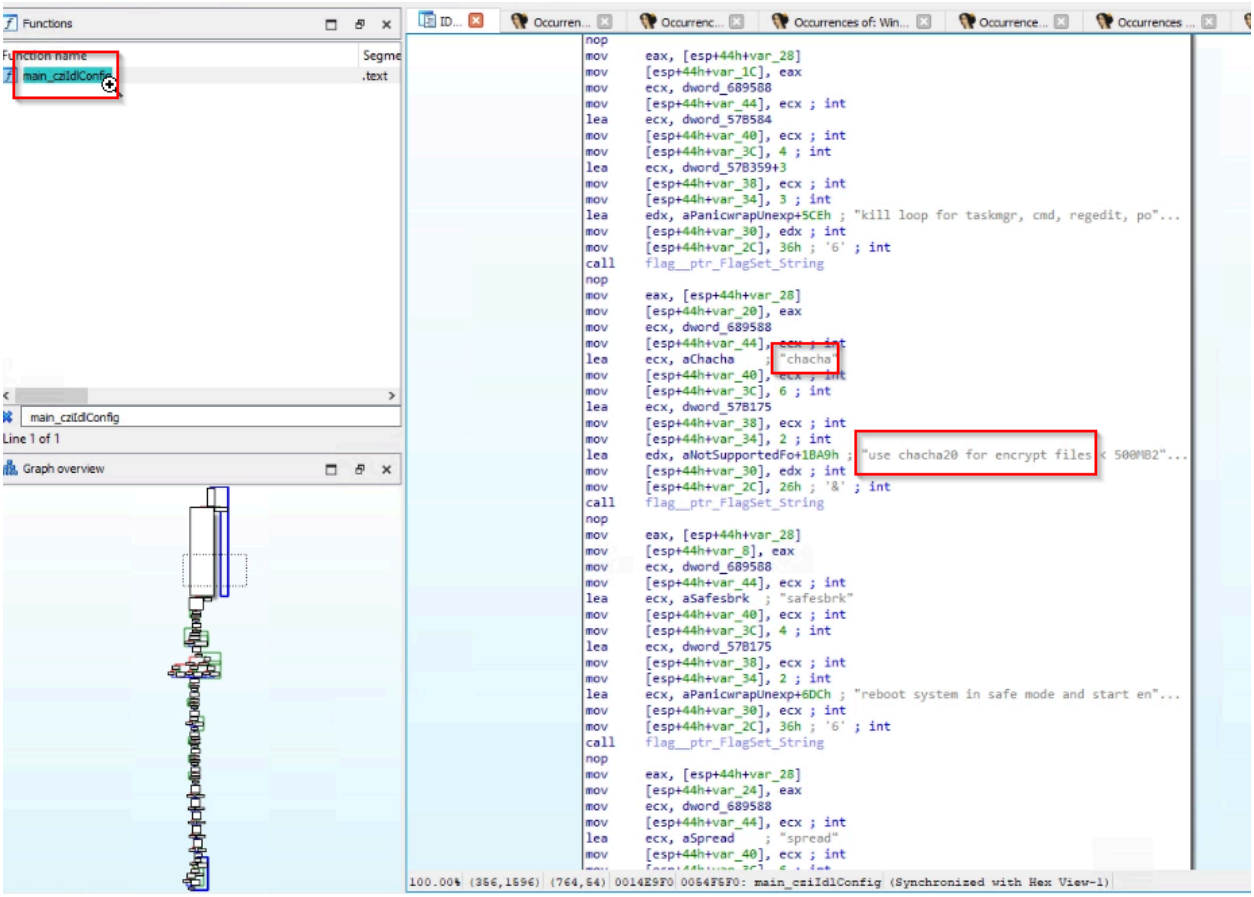
While investigating the `main_DisableWindowsDefender` function, I examined how the malware targets security components on the host operating system. As highlighted in the string configuration block, I found that the script dynamically loads the `MpClient.dll` module via `syscall/Load` to interact with Windows Defender APIs and disable its security features.



Answer: MpClient.dll

Task 5 : What is the decryption flag used by the malware?

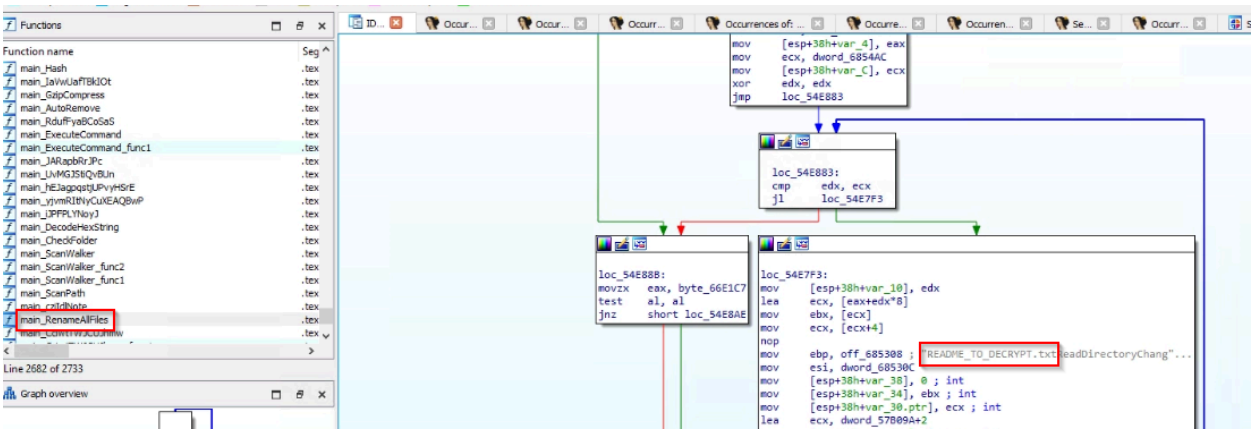
During my analysis of the cryptographic routines in the Golang binary, I inspected function strings and constants to identify the encryption algorithms used for locking victim files. As identified in the analysis modules, I confirmed that the malware relies on the `Chacha` stream cipher implementation for its file encryption operations.



Answer: Chacha

Task 6 : What is the file name of the ransomware note?

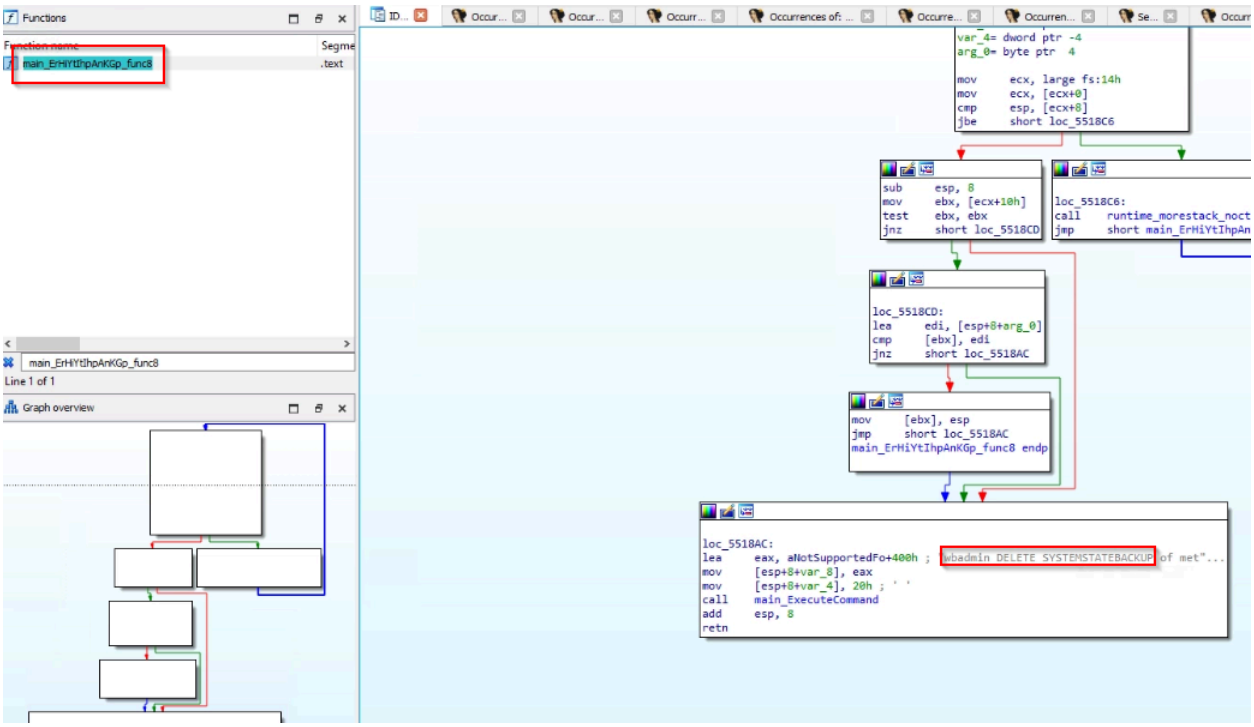
While analyzing the file-enumeration and post-encryption routine handled by `main_RenameAllFiles`, I checked the configuration for the ransom notification dropped into affected directories. As highlighted in the assembly instructions, I identified that the string references `README_TO_DECRYPT.txt` as the official ransom note filename.



Answer: README_TO_DECRYPT.txt

Task 7 : What command can be executed by the "main_ErHiYtlhpAnKGp_func8" function?

During my inspection on `main_ErHiYtlhpAnKGp_func8` function, As highlighted in the instruction block, I found that the function invokes `wbadmin DELETE SYSTEMSTATEBACKUP` to destroy local system backups and disable recovery options.



Answer: wbadmin DELETE SYSTEMSTATEBACKUP

Conclusion:

The analyzed Golang ransomware sample is a highly resilient, multi-threaded threat designed to evade traditional security controls, disable host-level defenses, and maximize system disruption through file encryption and backup destruction.

Upon execution, the malware initiates high-privilege routines—leveraging direct disk access (`\\.\PHYSICALDRIVE0`)—to prepare the environment and bypass standard access constraints. To ensure uninterrupted operation, it calls upon Windows Defender components (utilizing `MpClient.dll`) and executes embedded, base64-encoded PowerShell scripts designed to disable real-time monitoring (`Set-MpPreference -DisableRealtimeMonitoring $true`) while forcefully terminating competing processes and system utilities via `taskkill`.

For data destruction and obstruction of recovery, the threat invokes administrative routines like `wbadmin DELETE SYSTEMSTATEBACKUP` to wipe shadow copies and system backups. It then proceeds to traverse the file system, encrypting victim files utilizing the ChaCha stream cipher algorithm, and drops ransom instructions across target directories under the filename `README_TO_DECRYPT.txt`.